교육부 Ministry of Education

NRF 한국연구재단 National Research Foundation of Korea

공유교과목 [딥러닝]

케라스 기초

COSS Convergence and Open Sharing System 인공지능

전남대학교 CHONNAM NATIONAL UNIVERSITY

성균관대학교 SUNGKYUNKWAN UNIVERSITY

KNU 경북대학교 KYUNGPOOK NATIONAL UNIVERSITY

전주대학교 JEONJU UNIVERSITY

서울시립대학교 UNIVERSITY OF SEOUL

서울과학기술대학교 SEOUL NATIONAL UNIVERSITY OF SCIENCE & TECHNOLOGY

영진전문대학교 YEUNGJIN UNIVERSITY

Contents

- 신경망을 위한 데이터 표현 (텐서)
- 텐서 연산

신경망을 위한 데이터 표현

케라스 기초

- **신경망을 위한 데이터 표현**
  - 텐서 (Tensor)
    - ✓ 머신러닝의 기본 구성 요소
    - ✓ 데이터를 위한 컨테이너 (저장 공간)
    - ✓ 일반적으로 수치 데이터를 저장함
    - ✓ 임의의 차원 개수를 가지는 행렬의 일반화된 모습을 가짐

      : 텐서에서 차원 (dimension)은 일반적으로 축(axis)라고 부름

RGB와pixel in Tensor

  - 축에 따른 텐서의 종류 (numpy 패키지 예시)
    - ✓ 스칼라 : 0D 텐서
    - ✓ 벡터: 1D 텐서
    - ✓ 행렬 : 2D 텐서
    - ✓ 고차원 : 3D 텐서 및 기타 고차원 텐서

케라스 기초

- **신경망을 위한 데이터 표현**
  - 스칼라 (0D 텐서)
    - ✓ 하나의 숫자만 담고 있는 텐서를 스칼라(scalar) 라고 부름
    - ✓ 또는 스칼라 텐서, 0차원 텐서, 0D 텐서 등 다양하게 부를 수 있음
    - ✓ numpy에서는 float32나 float64 타입의 숫자가 스칼라 텐서에 해당함

스칼라 텐서의 예시는 다음과 같음

```
>>> import numpy as np
>>> x = np.array(12)
>>> x
array(12)
>>> x.ndim
0
```

스칼라 텐서의 축 개수는 0임(ndim == 0)

- **신경망을 위한 데이터 표현**
  - 행렬 (2D 텐서)
    - ✓ 벡터의 배열은 행렬(matrix) 또는 2D 텐서임
    - ✓ 행렬에는 2개의 축이 있음 (행(row)과 열(column)로 표기)
    - ✓ 행렬은 숫자가 채워진 사각 격자
    - ✓ 넘파이에서 행렬을 나타내면 다음과 같음

1열 5열

```
>>> x = np.array([[5, 78, 2, 34, 0],   ← 1행
                  [6, 79, 3, 35, 1],   ← 2행
                  [7, 80, 4, 36, 2]])  ← 3행
>>> x.ndim
2
```

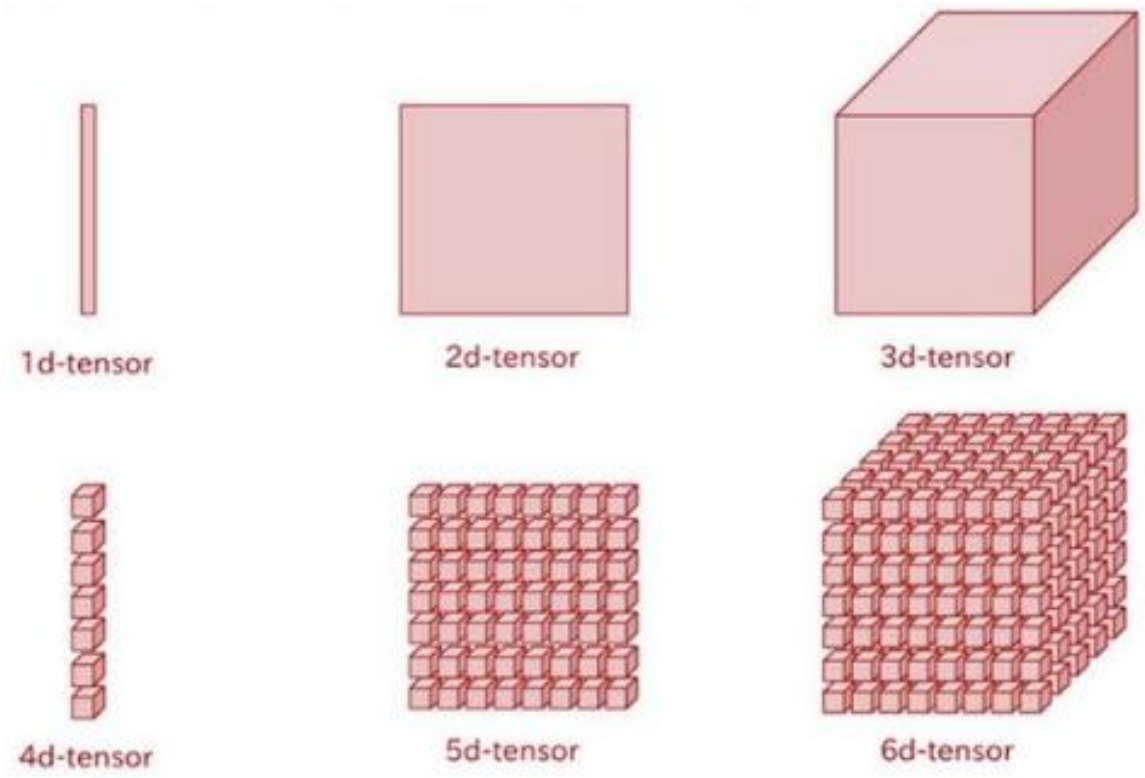
x의 첫 번째 행은 [5, 78, 2, 34, 0]이고, 첫 번째 열은 [5, 6, 7]임

- **신경망을 위한 데이터 표현**
  - 3D 텐서와 고차원 텐서
    - ✓ 아래 예시와 같이 행렬들을 한번 더 하나의 배열로 합치면 숫자가 채워진 직육면체 형태로 해석 가능한 3D 텐서가 만들어짐
    - ✓ Numpy에서 3D 텐서의 예시

```
>>> x = np.array([[[5, 78, 2, 34, 0],
                   [6, 79, 3, 35, 1],
                   [7, 80, 4, 36, 2]],
                  [[5, 78, 2, 34, 0],
                   [6, 79, 3, 35, 1],
                   [7, 80, 4, 36, 2]],
                  [[5, 78, 2, 34, 0],
                   [6, 79, 3, 35, 1],
                   [7, 80, 4, 36, 2]]])
>>> x.ndim
3
```

케라스 기초

- **신경망을 위한 데이터 표현**
  - 3D 텐서와 고차원 텐서
    - ✓ 같은 원리로 3D 텐서들을 하나의 배열로 합치면 4D 텐서 생성 가능
    - ✓ 딥러닝에서는 보통 0D에서 4D까지의 텐서를 다룸
    - ✓ 동영상 데이터를 다룰 경우에는 5D 텐서까지 활용 하기도 함



케라스 기초

- **신경망을 위한 데이터 표현**
  - 텐서 핵심 속성
    - ✓ 축의 개수 (랭크)
      - 예시: 0D, 2D, 3D …
    - ✓ 크기 (Shape)
      - 텐서의 각 축을 따라 얼마나 큰 데이터가 있는지를 나타냄
      - 예시 : 행렬의 크기 (3,5) / 3d텐서의 크기 (3,3,5) 등의 튜플 형태
    - ✓ 데이터 타입
      - 텐서에 포함된 데이터의 타입임
      - numpy에서는 dtype에 저장됨
      - 일반적으로 float32, uint8, float64등이 자주 활용되며 드물게 char 활용됨

※ 텐서는 사전에 할당되어 연속된 메모리에 저장되어야 하므로
넘파이 배열은 가변 길이의 문자열을 지원하지 않음

- **신경망을 위한 데이터 표현**
  - 예제 확인
    - ✓ MNIST 예제

training and test are different

```
from keras.datasets import mnist
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
```

```
>>> print(train_images.ndim)
3
```

train_images 배열의 ndim 속성으로 축의 개수를 확인함

```
>>> print(train_images.shape)
(60000, 28, 28)
```

배열의 크기 확인

```
>>> print(train_images.dtype)
uint8
```

데이터의 타입 확인

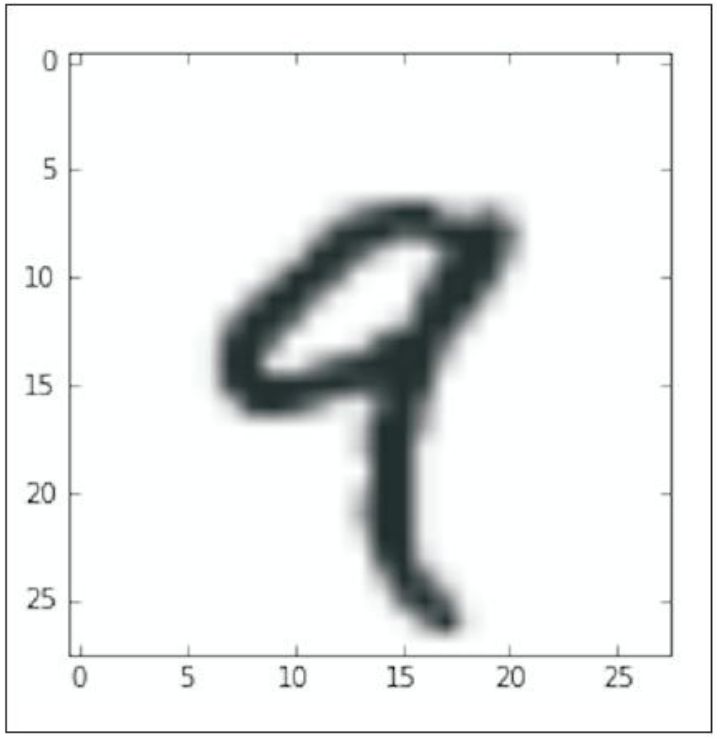
Mnist 데이터는 28×28 크기의 정수 행렬 6만 개의 배열

- **신경망을 위한 데이터 표현**
  - 예제 확인
    - ✓ MNIST 예제
    - ✓ Matplotlib 를 사용해서 실제 데이터를 확인

코드 2-6 다섯 번째 이미지 출력하기

```
digit = train_images[4]

import matplotlib.pyplot as plt
plt.imshow(digit, cmap=plt.cm.binary)
plt.show()
```



- **신경망을 위한 데이터 표현**
  - 예제 확인
    - ✓ Numpy로 텐서 조작
    - ✓ 슬라이싱 (Slicing)을 통해 아래와 같이 여러 개의 데이터를 선택 가능

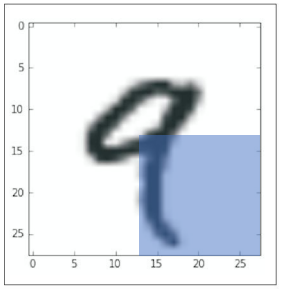
```
>>> my_slice = train_images[10:100]
>>> print(my_slice.shape)
(90, 28, 28)
```

- ✓ :(콜론)은 전체 인덱스를 선택함

```
>>> my_slice = train_images[10:100, :, :] -------- 이전 예와 동일합니다.
>>> my_slice.shape
(90, 28, 28)
>>> my_slice = train_images[10:100, 0:28, 0:28] -------- 역시 이전과 동일합니다.
>>> my_slice.shape
(90, 28, 28)
```

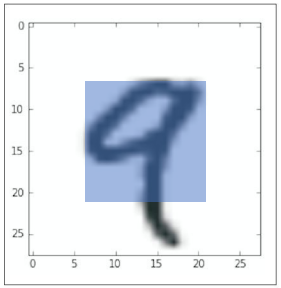
- **신경망을 위한 데이터 표현**
  - 예제 확인
    - ✓ Numpy로 텐서 조작
    - ✓ 각 배열의 축을 따라 인덱스 사이도 선택 가능
      - Ex. 이미지의 오른쪽 아래 14 x 14 픽셀을 아래와 같이 선택 가능

```
my_slice = train_images[:, 14:, 14:]
```



    - ✓ 음수 인텍스를 통해 각축의 끝에서의 상대적 위치 인덱싱 가능
      - 정중앙에 위치한 14×14픽셀 조각을 이미지에서 잘라내기

```
my_slice = train_images[:, 7:-7, 7:-7]
```



케라스 기초

- **신경망을 위한 데이터 표현**
  - 배치 (batch) 데이터 load data mass
    - ✓ 일반적으로 딥러닝에서 사용하는 모든 데이터 텐서의 첫 번째 축은 샘플 축
    - ✓ 딥러닝 모델은 한 번에 전체 데이터셋을 처리하지 않으며 작은 배치(batch)로 나눔
    - ✓ MNIST 숫자 데이터에서 크기가 128인 배치 하나는 다음과 같음

✓ `batch = train_images[:128]` ← 첫번째 batch

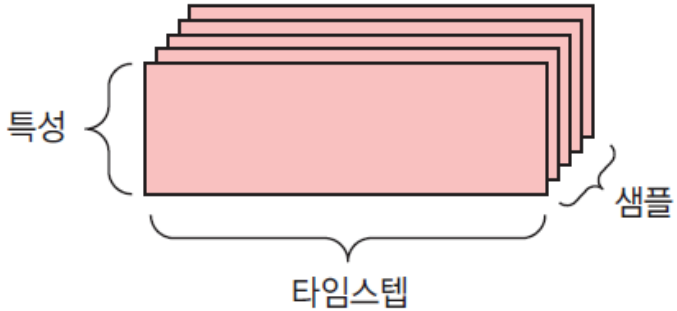
`batch = train_images[128:256]` ← 두번째 batch

✓ 이런 배치 데이터를 나눌 때는 첫 번째 축(0번 축)을 배치 축(batch axis) 또는 배치 차원(batch dimension)이라고 부름

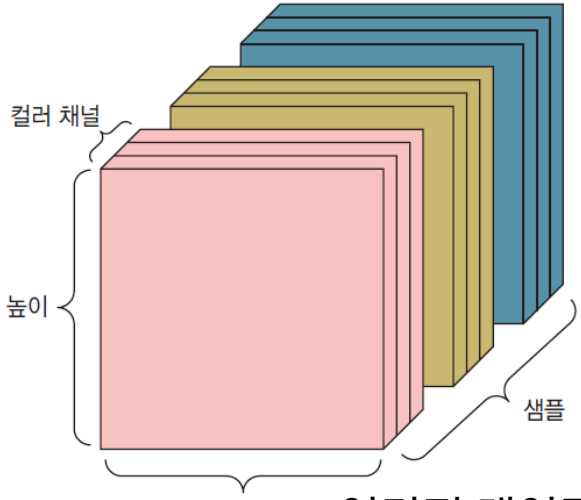
- **신경망을 위한 데이터 표현**
  - 텐서의 실제사례
    : 실제로 우리가 분석하고 다루는 정형/비정형 데이터의 종류

종류

✓ 벡터 데이터 : (samples, features) – 2D 텐서
✓ 시계열 데이터 : (samples, timesteps, features) – 3D 텐서 2, 300, 1
✓ 이미지 : (samples, channels, height, width) – 4D 텐서
✓ 동영상 : (samples, frames, channels, height, width) – 5D 텐서



[시계열 데이터예시]



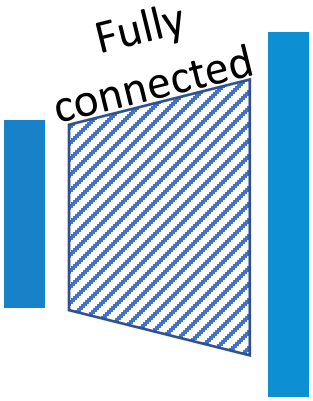
[이미지 데이터예시]

텐서 연산

- **텐서 연산 (numpy)**
  - 심층 신경망이 학습한 모든 변환은 수치 데이터 텐서에 적용하는 텐서 연산(tensor operation)으로 나타낼 수 있음
  - 예를 들어 텐서 덧셈이나 텐서 곱셈 등임

  - 아래 예제를 통해 확인
    - ✓ Dense 층을 쌓아서 신경망 생성
    - ✓ 케라스에서의 신경망 층은 다음과 같이 생성
    - ✓ 512개의 출력층을 가진 dense layer 생성 (활성화 함수 relu)

```
keras.layers.Dense(512, activation='relu')
```



입력층 : 임의의 차원

출력층 : 512 차원

케라스 기초

- **텐서 연산 (numpy)**
  - 아래 예제를 통해 확인
    - ✓ Dense 층을 쌓아서 신경망 생성
    - ✓ 케라스에서의 신경망 층은 다음과 같이 초기화 및 생성 가능함
      (활성화 함수: relu / 층의 크기: 512 차원 (w,b)를 자동으로 초기화)

$y = \sigma(w^T x)$

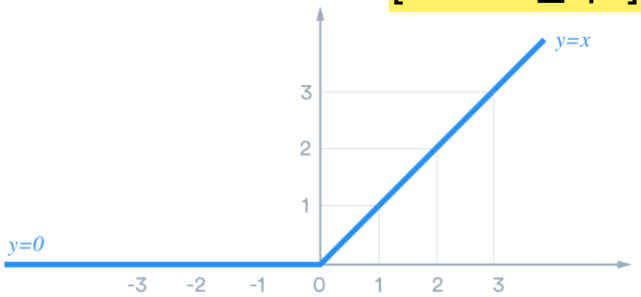
```
keras.layers.Dense(512, activation='relu')
```

- ✓ 위 초기화 함수는 아래와 같은 동작을 수행함

```
output = relu(dot(W, input) + b)
```

가중치 W는 2D 텐서, bias b는 벡터
이들은 모두 신경망의 구성 요소임

[Relu 함수]



- ✓ 실제로 3개의 텐서 연산이 포함됨
  : dot product, 덧셈, relu(x) 함수 연산
  : relu(x) 함수는 max(x,0)으로 동작함 그래프 참조

$y = \mathrm{Relu}(W^T x + b)$

케라스 기초

- **텐서 연산 (numpy)**
  - 원소별 연산
    - ✓ relu 함수와 덧셈은 원소별 연산(element-wise operation)
    - ✓ 해당 연산은 텐서에 있는 각 원소에 독립적으로 적용
    - ✓ 파이썬 에서는 for 반복문이용하여 원소별 연산을 구현 가능

```
def naive_relu(x):
    assert len(x.shape) == 2 -------- x는 2D 넘파이 배열입니다.

    x = x.copy() -------- 입력 텐서 자체를 바꾸지 않도록 복사합니다.
    for i in range(x.shape[0]):
        for j in range(x.shape[1]):
            x[i, j] = max(x[i, j], 0)
    return x
```

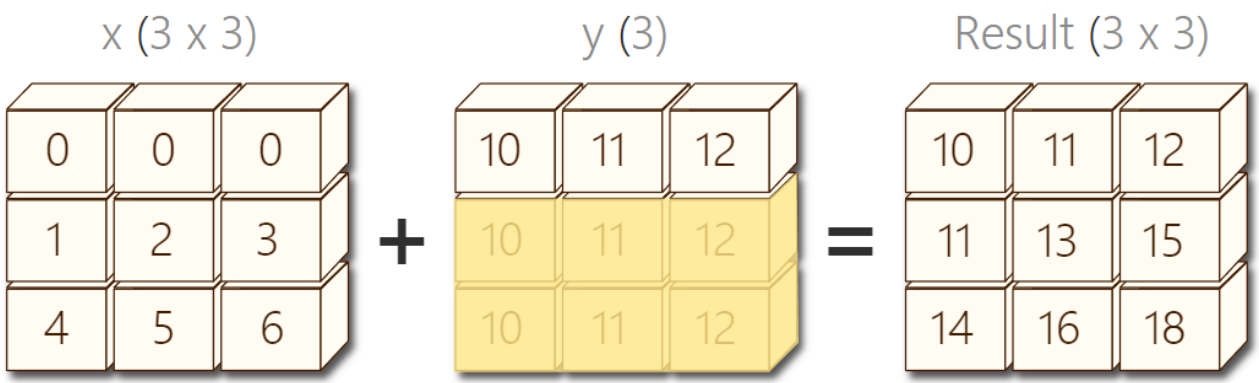
assert: https://wikidocs.net/21050 x.copy: https://blockdmask.tistory.com/576

- **텐서 연산 (numpy)**
  - 원소별 연산
    - ✓ 사칙연산 (덧셈,뺄셈 곱셈 등)도 동일하게 구현 가능
    - ✓ 아래 코드 참조

```
def naive_add(x, y):
    assert len(x.shape) == 2 -------- x와 y는 2D 넘파이 배열입니다.
    assert x.shape == y.shape

    x = x.copy() -------- 입력 텐서 자체를 바꾸지 않도록 복사합니다.
    for i in range(x.shape[0]):
        for j in range(x.shape[1]):
            x[i, j] += y[i, j]
    return x
```

- **텐서 연산 (numpy)**
  - 브로드캐스팅
    - ✓ 서로 크기가 다른 텐서들간의 연산시 필요
    - ✓ 크기가 다른 두 텐서가 더해질 때 작은 텐서가 큰 텐서의 크기에 맞추어 브로드캐스팅(broadcasting) 됨
    - ✓ 이는 다음 두 단계를 통해 수행됨
      - 큰 텐서의 ndim에 맞도록 작은 텐서에 (브로드캐스팅 축이라고 부르는) 축이 추가됨
      - 작은 텐서가 새 축을 따라서 큰 텐서의 크기에 맞도록 반복됨



[텐서 브로드캐스팅 예시]

- **텐서 연산 (numpy)**
  - 브로드캐스팅
    - ✓ 구현 예시 1

```
def naive_add_matrix_and_vector(x, y):
    assert len(x.shape) == 2 -------- x는 2D 넘파이 배열입니다.
    assert len(y.shape) == 1 -------- y는 넘파이 벡터입니다.
    assert x.shape[1] == y.shape[0]

    x = x.copy() -------- 입력 텐서 자체를 바꾸지 않도록 복사합니다.
    for i in range(x.shape[0]):
        for j in range(x.shape[1]):
            x[i, j] += y[j]
    return x
```

- **텐서 연산 (numpy)**
  - 브로드캐스팅
    - ✓ 구현 예시 2
      - (a, b, .. . n, n + 1, .. . m) 크기의 텐서와 (n, n + 1, .. . m) 크기의 텐서 사이에 브로드캐스팅으로 원소별 연산 가능
      - 브로드캐스팅은 a부터 n – 1까지의 축에 자동으로 일어남
      - 아래 코드는 크기가 다른 두 텐서에 maximum 연산을 수행 (브로드캐스팅)

```
import numpy as np

x = np.random.random((64, 3, 32, 10)) -------- x는 (64, 3, 32, 10) 크기의 랜덤 텐서입니다.
y = np.random.random((32, 10)) -------- y는 (32, 10) 크기의 랜덤 텐서입니다.

z = np.maximum(x, y) -------- 출력 z 크기는 x와 동일하게 (64, 3, 32, 10)입니다.
```

케라스 기초

- **텐서 연산 (numpy)**
  - Dot product (점곱 연산)
    - ✓ 원소별 연산과 반대로 입력 텐서의 원소들을 결합시킴
    - ✓ 넘파이, 케라스, 씨아노, 텐서플로에서 원소별 곱셈은 * 연산자를 사용함
    - ✓ 텐서플로에서는 dot 연산자가 다르지만
      넘파이와 케라스는 점곱 연산에 보편적인 dot 연산자를 사용함

```
import numpy as np
z = np.dot(x, y)
```

- **텐서 연산 (numpy)**
  - Dot product (점곱 연산)
    - ✓ 예시 : 2개의 벡터 x,y간의 dot product 수행
    - ✓ 연산 결과물 : 스칼라

```
def naive_vector_dot(x, y):
    assert len(x.shape) == 1
    assert len(y.shape) == 1
    assert x.shape[0] == y.shape[0]

    z = 0.
    for i in range(x.shape[0]):
        z += x[i] * y[i]
    return z
```

x와 y는 넘파이 벡터입니다.

- **텐서 연산 (numpy)**
  - Dot product (점곱 연산)
    - ✓ 예시 : 행렬x와 벡터y 간의 dot product 수행
    - ✓ 연산 결과물 : 벡터

```
import numpy as np
def naive_matrix_vector_dot(x, y):
    assert len(x.shape) == 2 -------- x는 넘파이 행렬입니다.
    assert len(y.shape) == 1 -------- y는 넘파이 벡터입니다.
    assert x.shape[1] == y.shape[0] -------- x의 두 번째 차원이 y의 첫 번째 차원과 같아야 합니다!

    z = np.zeros(x.shape[0]) -------- 이 연산은 x의 행과 같은 크기의 0이 채워진 벡터를 만듭니다.
    for i in range(x.shape[0]):
        for j in range(x.shape[1]):
            z[i] += x[i, j] * y[j]
    return z
```

- **텐서 연산 (numpy)**
  - Dot product (점곱 연산)
    - ✓ 두 텐서중 하나라도 ndim이 1보다 큰경우에는 dot product 는 교환법칙이성립하지 않는다.
    - ✓ 즉, dot (x,y) ≠ dot (y,x), 아래 코드 참조

```
def naive_matrix_vector_dot(x, y):
    z = np.zeros(x.shape[0])
    for i in range(x.shape[0]):
        z[i] = naive_vector_dot(x[i, :], y)
    return z
```

- **텐서 연산 (numpy)**
  - Dot product (점곱 연산)
    - ✓ 예시 : 두 행렬간의 dot product
    - ✓ 가장 일반화된 dot product 활용

```
def naive_matrix_dot(x, y):
    assert len(x.shape) == 2
    assert len(y.shape) == 2
    assert x.shape[1] == y.shape[0]

    z = np.zeros((x.shape[0], y.shape[1]))
    for i in range(x.shape[0]):
        for j in range(y.shape[1]):
            row_x = x[i, :]
            column_y = y[:, j]
            z[i, j] = naive_vector_dot(row_x, column_y)
    return z
```

x와 y는 넘파이 행렬입니다.

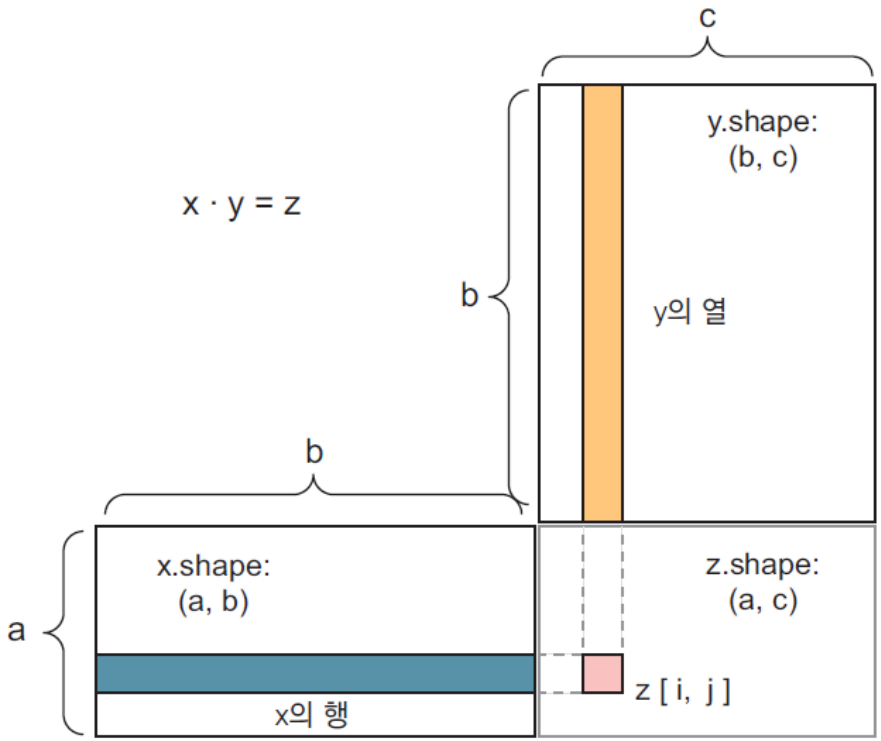
x의 두 번째 차원이 y의 첫 번째 차원과 같아야 합니다!

이 연산은 0이 채워진 특정 크기의 벡터를 만듭니다.

x의 행을 반복합니다.

y의 열을 반복합니다.

- **텐서 연산 (numpy)**
  - Dot product (점곱 연산)
    - ✓ 아래 그림 1. 를 통해 dot product 의 연산에 대해 이해
    - ✓ 3차원 이상의 고차원 텐서간의 dot product 또한 그림 2와 같이 수행 가능



[그림 1. dot product 이해]

(a, b, c, d) · (d,) → (a, b, c)

(a, b, c, d) · (d, e) → (a, b, c, e)

[그림 2. 고차원 텐서의 dot product]

- **텐서 연산 (numpy)**
  - 텐서의 크기 변환 (reshape)
    - ✓ 특정 크기에 맞게 텐서의 열과 행을 재배열
    - ✓ 변환된 텐서는 기존 텐서와 원소 개수가 동일함

```
>>> x = np.array([[0., 1.],
                  [2., 3.],
                  [4., 5.]])
>>> print(x.shape)
(3, 2)
>>> x = x.reshape((6, 1))
>>> x
array([[ 0.],
       [ 1.],
       [ 2.],
       [ 3.],
       [ 4.],
       [ 5.]])
>>> x = x.reshape((2, 3))
>>> x
```

```
array([[ 0.,  1.,  2.],
       [ 3.,  4.,  5.]])
```

- **텐서 연산 (numpy)**
  - 텐서의 크기 변환 (Transpose)
    - ✓ 전치: 행렬의 전치는 행과 열을 바꿈
    - ✓ Ex) x[i, :] (전치) → x[:, i]

```
>>> x = np.zeros((300, 20)) -------- 모두 0으로 채워진 (300, 20) 크기의 행렬을 만듭니다.
>>> x = np.transpose(x)
>>> print(x.shape)
(20, 300)
```

케라스 기초

- **텐서 생성 (tensorflow)**
  - 텐서 값 초기화 (상수)
    - ✓ numpy 과 생성 및 활용 방법이 유사하며, 호환도 가능함
    - ✓ Constant 텐서는 초기화 이후 값을 변경할 수 없음
    - ✓ 아래와 같이 텐서 생성 가능 (예시 2x2)

```
import tensorflow as tf
x = tf.constant([[5, 2], [1, 3]])
print(x)
```

```
tf.Tensor(
[[5 2]
 [1 3]], shape=(2, 2), dtype=int32)
```

생성된 2x2 텐서

```
x.numpy()
```

텐서 → numpy 변환

```
array([[5, 2],
       [1, 3]], dtype=int32)
```

- **텐서 생성 (tensorflow)**
  - 텐서 확인
    - ✓ numpy 과 생성 및 활용 방법이 유사하며, 호환도 가능함
    - ✓ 데이터의 타입과 모양 확인은 아래와 같이 수행

```
print("dtype:", x.dtype)
print("shape:", x.shape)
```

```
dtype: <dtype: 'int32'>
shape: (2, 2)
```

케라스 기초

- **텐서 생성 (tensorflow)**
  - 텐서 값 초기화
    - ✓ numpy 과 생성 및 활용 방법이 유사하며, 호환도 가능함
    - ✓ 0 또는 1로 구성된 벡터, 행렬 초기화 또한 numpy와 유사하게 가능

```
print(tf.ones(shape=(2, 1)))
print(tf.zeros(shape=(2, 1)))
```

```
tf.Tensor(
[[1.]
 [1.]], shape=(2, 1), dtype=float32)
tf.Tensor(
[[0.]
 [0.]], shape=(2, 1), dtype=float32)
```

케라스 기초

- **텐서 생성 (tensorflow)**
  - 텐서 값 초기화
    - ✓ numpy 과 생성 및 활용 방법이 유사하며, 호환도 가능함
    - ✓ 마찬가지로, 아래와 같은 표준 정규 분포 & uniform 분포 등을 따르는 난수 초기화 가능

```
x = tf.random.normal(shape=(2, 2), mean=0.0, stddev=1.0)

x = tf.random.uniform(shape=(2, 2), minval=0, maxval=10, dtype="int32")
```

    - ✓ np.random.normal
    - ✓ np.random.uniform 등의 난수 발생 함수와 활용이 대부분 유사함

- **텐서 생성 (tensorflow)**
  - 텐서 변수
    - ✓ 네트워크의 파라미터 (weight, bias)는 학습 과정중 값 업데이트가 필요
    - ✓ 값이 변하는 텐서는 변수: Variable로 지정이 필요
      - Tf.Variable() 함수 활용

```
initial_value = tf.random.normal(shape=(2, 2))
a = tf.Variable(initial_value)
print(a)
```

```
<tf.Variable 'Variable:0' shape=(2, 2) dtype=float32, numpy=
array([[-1.7639292,  0.4263797],
       [-0.3954156, -0.6072024]], dtype=float32)>
```

- 텐서 생성 (tensorflow)
  - 텐서 값 할당
    - ✓ 필요에 따라 assign(할당값), assign_add(더하는 값), assign_sub(빼는 값) 를 이용하여 값 할당

```
new_value = tf.random.normal(shape=(2, 2))
a.assign(new_value)
for i in range(2):
    for j in range(2):
        assert a[i, j] == new_value[i, j]

added_value = tf.random.normal(shape=(2, 2))
a.assign_add(added_value)
for i in range(2):
    for j in range(2):
        assert a[i, j] == new_value[i, j] + added_value[i, j]
```

케라스 기초

- **텐서 생성 (tensorflow)**
  - 텐서 연산
    - ✓ tf.math 패키지에서도 연산 기능 확인
    - ✓ https://www.tensorflow.org/api_docs/python/tf/math

```
a = tf.random.normal(shape=(2, 2))
b = tf.random.normal(shape=(2, 2))

c = a + b
d = tf.square(c)
e = tf.exp(d)
```

사칙연산

다양한 수학 연산

- 텐서 생성 (tensorflow)
  - 텐서 연산
    - ✓ tf.math 패키지에서도 연산 기능 확인
    - ✓ https://www.tensorflow.org/api_docs/python/tf/math

`abs(...)` : Computes the absolute value of a tensor.

`accumulate_n(...)` : Returns the element-wise sum of a list of tensors.

`acos(...)` : Computes acos of x element-wise.

`acosh(...)` : Computes inverse hyperbolic cosine of x element-wise.

`add(...)` : Returns x + y element-wise.

`add_n(...)` : Adds all input tensors element-wise.

`angle(...)` : Returns the element-wise argument of a complex (or real) tensor.

`argmax(...)` : Returns the index with the largest value across axes of a tensor.

`argmin(...)` : Returns the index with the smallest value across axes of a tensor.

`asin(...)` : Computes the trignometric inverse sine of x element-wise.

`asinh(...)` : Computes inverse hyperbolic sine of x element-wise.

`atan(...)` : Computes the trignometric inverse tangent of x element-wise.

`atan2(...)` : Computes arctangent of `y/x` element-wise, respecting signs of the arguments.

`atanh(...)` : Computes inverse hyperbolic tangent of x element-wise.

`softmax(...)` : Computes softmax activations.

`softplus(...)` : Computes elementwise softplus: `softplus(x) = log(exp(x) + 1)`.

`softsign(...)` : Computes softsign: `features / (abs(features) + 1)`.

`sqrt(...)` : Computes element-wise square root of the input tensor.

`square(...)` : Computes square of x element-wise.

`squared_difference(...)` : Returns conj(x - y)(x - y) element-wise.

`subtract(...)` : Returns x - y element-wise.

`tan(...)` : Computes tan of x element-wise.

`tanh(...)` : Computes hyperbolic tangent of `x` element-wise.

`top_k(...)` : Finds values and indices of the `k` largest entries for the last dimension.

`truediv(...)` : Divides x / y elementwise (using Python 3 division operator semantics).

`unsorted_segment_max(...)` : Computes the maximum along segments of a tensor.

`unsorted_segment_mean(...)` : Computes the mean along segments of a tensor.

`unsorted_segment_min(...)` : Computes the minimum along segments of a tensor.

`unsorted_segment_prod(...)` : Computes the product along segments of a tensor.

`unsorted_segment_sqrt_n(...)` : Computes the sum along segments of a tensor divided by the sqrt(N).

`unsorted_segment_sum(...)` : Computes the sum along segments of a tensor.

`xdivy(...)` : Returns 0 if x == 0, and x / y otherwise, elementwise.

`xlog1py(...)` : Compute x * log1p(y).

`xlogy(...)` : Returns 0 if x == 0, and x * log(y) otherwise, elementwise.

`zero_fraction(...)` : Returns the fraction of zeros in `value`.

`zeta(...)` : Compute the Hurwitz zeta function $\zeta(x, q)$.

케라스 기초

- **텐서 생성 (tensorflow)**
  - Layer 생성 예시
    - ✓ 아래와 같이 직접 Linear라는 클래스 이름으로 레이어 설계 가능

```
class Linear(keras.layers.Layer):
    """y = w.x + b"""

    def __init__(self, units=32, input_dim=32):
        super(Linear, self).__init__()
        w_init = tf.random_normal_initializer()
        self.w = tf.Variable(
            initial_value=w_init(shape=(input_dim, units), dtype="float32"),
            trainable=True,
        )
        b_init = tf.zeros_initializer()
        self.b = tf.Variable(
            initial_value=b_init(shape=(units,), dtype="float32"), trainable=True
        )

    def call(self, inputs):
        return tf.matmul(inputs, self.w) + self.b
```

케라스 기초

- **텐서 생성 (tensorflow)**
  - Layer 생성 예시
    - ✓ 설계한 레이어를 초기화 (객체 생성)
    - ✓ 레이어의 구성요소 확인 및 파라미터 값 (weight, bias) 접근

```
# Instantiate our layer.
linear_layer = Linear(units=4, input_dim=2)

# The layer can be treated as a function.
# Here we call it on some data.
y = linear_layer(tf.ones((2, 2)))
assert y.shape == (2, 4)

assert linear_layer.weights == [linear_layer.w, linear_layer.b]
```

교육부 Ministry of Education

NRF 한국연구재단 National Research Foundation of Korea

thank you

본 과제(결과물)는 교육부와 한국연구재단의 재원으로 지원을 받아 수행된
디지털신기술인재양성 혁신공유대학사업의 연구결과입니다.

Convergence and Open Sharing System COSS 인공지능 · 전남대학교 CHONNAM NATIONAL UNIVERSITY · 성균관대학교 SUNGKYUNKWAN UNIVERSITY · KNU 경북대학교 KYUNGPOOK NATIONAL UNIVERSITY · 전주대학교 JEONJU UNIVERSITY · 서울시립대학교 UNIVERSITY OF SEOUL · 서울과학기술대학교 SEOUL NATIONAL UNIVERSITY OF SCIENCE & TECHNOLOGY · 영진전문대학교 YEUNGJIN UNIVERSITY